

《漏洞利用及渗透测试基础》实验报告

姓名：王一诺 学号：2312331 班级：计科二班

实验名称：

AFL 模糊测试

实验要求：

根据课本 7.4.5 章节,复现 AFL 在 KALI 下的安装、应用,查阅资料理解覆盖引导和文件变异的概念和含义。

实验过程：

1. 安装 AFL

进入 vmware, 打开 kali-linux-2021.1 虚拟机, 右键选择打开终端, 输入命令行 `wget http://lcamtuf.coredump.cx/afl/releases/afl-latest.tgz` 来下载 AFL-2.52b 安装包, 如下图所示:

```
(kali@kali)~[~/Desktop]
$ wget http://lcamtuf.coredump.cx/afl/releases/afl-latest.tgz
--2025-04-22 01:27:52-- http://lcamtuf.coredump.cx/afl/releases/afl-latest.tgz
正在解析主机 lcamtuf.coredump.cx (lcamtuf.coredump.cx) ... 172.67.180.222, 184.21.83.192, 2606:4700:3036::ac43:b4de, ...
正在连接 lcamtuf.coredump.cx (lcamtuf.coredump.cx)|172.67.180.222|:80... 已连接。
已发出 HTTP 请求, 正在等待响应 ... 301 Moved Permanently
位置: https://lcamtuf.coredump.cx/afl/releases/afl-latest.tgz [跟随至新的 URL]
--2025-04-22 01:27:53-- https://lcamtuf.coredump.cx/afl/releases/afl-latest.tgz
正在连接 lcamtuf.coredump.cx (lcamtuf.coredump.cx)|172.67.180.222|:443... 已连接。
已发出 HTTP 请求, 正在等待响应 ... 200 OK
长度: 835907 (816K) [application/x-gzip]
正在保存至: "afl-latest.tgz"

afl-latest.tgz 100%[=====>] 816.32K 907KB/s 用时 0.9s

2025-04-22 01:27:55 (907 KB/s) - 已保存 "afl-latest.tgz" (835907/835907))
```

然后我们输入命令行 `tar xvf afl-latest.tgz`, 解压安装包, 如下图所示:

```
(kali@kali)~[~/Desktop]
$ tar xvf afl-latest.tgz
afl-2.52b/
afl-2.52b/afl-as.h
afl-2.52b/libtokencap/
afl-2.52b/libtokencap/libtokencap.so.c
afl-2.52b/libtokencap/README.tokencap
afl-2.52b/libtokencap/Makefile
afl-2.52b/alloc-inl.h
afl-2.52b/config.h
afl-2.52b/test-instr.c
afl-2.52b/afl-analyze.c
afl-2.52b/README
afl-2.52b/afl-showmap.c
afl-2.52b/experimental/
afl-2.52b/experimental/clang_asm_normalize/
afl-2.52b/experimental/clang_asm_normalize/as
afl-2.52b/experimental/README.experiments
afl-2.52b/experimental/distributed_fuzzing/
afl-2.52b/experimental/distributed_fuzzing/sync_script.sh
afl-2.52b/experimental/crash_triage/
afl-2.52b/experimental/crash_triage/triage_crashes.sh
afl-2.52b/experimental/libpng_no_checksum/
afl-2.52b/experimental/libpng_no_checksum/libpng-nocrc.patch
afl-2.52b/experimental/bash_shellshock/
afl-2.52b/experimental/bash_shellshock/shellshock-fuzz.diff
afl-2.52b/experimental/canvas_harness/canvas_harness.html
afl-2.52b/experimental/asan_cgroups/
afl-2.52b/experimental/asan_cgroups/limit_memory.sh
afl-2.52b/experimental/post_library/
afl-2.52b/experimental/post_library/post_library.so.c
afl-2.52b/experimental/post_library/post_library_png.so.c
afl-2.52b/experimental/persistent_demo/
afl-2.52b/experimental/persistent_demo/persistent_demo.c
afl-2.52b/experimental/argv_fuzzing/
afl-2.52b/experimental/argv_fuzzing/argv-fuzz-inl.h
afl-2.52b/afl-cmin
afl-2.52b/afl-gotcpu.c
afl-2.52b/libdislocator/
afl-2.52b/libdislocator/README.dislocator
afl-2.52b/libdislocator/Makefile
afl-2.52b/libdislocator/libdislocator.so.c
afl-2.52b/docs/
afl-2.52b/docs/perf_tips.txt
afl-2.52b/docs/historical_notes.txt
afl-2.52b/docs/COPYING
afl-2.52b/docs/status_screen.txt
afl-2.52b/docs/sister_projects.txt
afl-2.52b/docs/vuln_samples/
afl-2.52b/docs/vuln_samples/strings-bfd-badptr.clf
afl-2.52b/docs/vuln_samples/ffmpeg-h264-call-stack-overflow.mp4
afl-2.52b/docs/vuln_samples/libtiff-uninit-mem4.tif
afl-2.52b/docs/vuln_samples/firefox-bmp-leak.bmp
afl-2.52b/docs/vuln_samples/sqlite-null-ptr3.sql
afl-2.52b/docs/vuln_samples/ffmpeg-h264-bad-ptr-800m.mp4
afl-2.52b/docs/vuln_samples/msie-png-mem-leak.png
afl-2.52b/docs/vuln_samples/sqlite-bad-ptr.sql
afl-2.52b/docs/vuln_samples/libtiff-uninit-mem.tif
afl-2.52b/docs/vuln_samples/jxrlib-crash4.jxr
afl-2.52b/docs/vuln_samples/unzip-t-mem-corruption.zip
```

这时会出现一个 afl-2.52b 文件夹, 输入命令行 `cd afl-2.52b` 使终端在对应的文件夹运行。然后输入 `sudo make && sudo make install` 编译 AFL, 如下图所示:

```

(kali@kali) [~/Desktop]
$ cd afl-2.52b

(kali@kali) [~/Desktop/afl-2.52b]
$ sudo make 66 sudo make install

[sudo] kali 的密码:
[*] Checking for the ability to compile x86 code...
[*] Everything seems to be working, ready to compile.
cc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" afl-gcc.c -o afl-gcc -ldl
set -e; for i in afl-g++ afl-clang afl-clang++; do ln -sf afl-gcc $i; done
cc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" afl-fuzz.c -o afl-fuzz -ldl
cc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" afl-showmap.c -o afl-showmap -ldl
cc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" afl-tmin.c -o afl-tmin -ldl
cc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" afl-gotcpu.c -o afl-gotcpu -ldl
cc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" afl-analyze.c -o afl-analyze -ldl
cc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" afl-as.c -o afl-as -ldl
ln -sf afl-as as

[*] Testing the CC wrapper and instrumentation output...
unset AFL_USE_ASAN AFL_USE_MSAN; AFL_QUIET=1 AFL_INST_RATIO=100 AFL_PATH="/usr/local/lib/afl/" -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" test-instr.c -o test-instr -ldl
echo 0 | ./afl-showmap -m none -q -o .test-instr ./test-instr
echo 1 | ./afl-showmap -m none -q -o .test-instr ./test-instr
[*] All right, the instrumentation seems to be working!
[*] LLVM users: see llvm_mode/README.llvm for a faster alternative to afl-gcc.
[*] All done! Be sure to review README - it's pretty short and useful.

[*] Checking for the ability to compile x86 code...
[*] Everything seems to be working, ready to compile.
[*] Testing the CC wrapper and instrumentation output...
unset AFL_USE_ASAN AFL_USE_MSAN; AFL_QUIET=1 AFL_INST_RATIO=100 AFL_PATH="/usr/local/lib/afl/" -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" test-instr.c -o test-instr -ldl
echo 0 | ./afl-showmap -m none -q -o .test-instr ./test-instr
echo 1 | ./afl-showmap -m none -q -o .test-instr ./test-instr
[*] All right, the instrumentation seems to be working!
[*] LLVM users: see llvm_mode/README.llvm for a faster alternative to afl-gcc.
[*] All done! Be sure to review README - it's pretty short and useful.

mkdir -p -m 755 ${DESTDIR}/usr/local/bin ${DESTDIR}/usr/local/lib/afl ${DESTDIR}/usr/local/share/doc/afl ${DESTDIR}/usr/local/share/afl
rm -f ${DESTDIR}/usr/local/bin/afl-plot.sh
install -m 755 afl-gcc afl-fuzz afl-showmap afl-tmin afl-gotcpu afl-analyze afl-plot afl-cmin afl-whatsup ${DESTDIR}/usr/local/bin
rm -f ${DESTDIR}/usr/local/bin/afl-as
if [ -f afl-gemu-trace ]; then install -m 755 afl-gemu-trace ${DESTDIR}/usr/local/bin; fi
if [ -f afl-clang-fast -a -f afl-llvm-pass.so -a -f afl-llvm-rt.o ]; then set -e; install -m 755 afl-clang-fast ${DESTDIR}/usr/local/bin; ln -sf afl-clang-fast ${DESTDIR}/usr/local/bin/afl-clang-fast; install -m 755 afl-llvm-pass.so afl-llvm-rt.o ${DESTDIR}/usr/local/lib/afl; fi
if [ -f afl-llvm-rt-32.o ]; then set -e; install -m 755 afl-llvm-rt-32.o ${DESTDIR}/usr/local/lib/afl; fi
if [ -f afl-llvm-rt-64.o ]; then set -e; install -m 755 afl-llvm-rt-64.o ${DESTDIR}/usr/local/lib/afl; fi
set -e; for i in afl-g++ afl-clang afl-clang++; do ln -sf afl-gcc ${DESTDIR}/usr/local/bin/$i; done
install -m 755 afl-as ${DESTDIR}/usr/local/lib/afl
ln -sf afl-as ${DESTDIR}/usr/local/lib/afl/as
install -m 644 docs/README docs/ChangeLog docs/*.txt ${DESTDIR}/usr/local/share/doc/afl
cp -r testcases/ ${DESTDIR}/usr/local/share/afl
cp -r dictionaries/ ${DESTDIR}/usr/local/share/afl

```

输入 `ls /usr/local/bin/afl*` 查看 afl 安装的文件，可见安装成功，如下图所示：

```

(kali@kali) [~/Desktop/afl-2.52b]
$ ls /usr/local/bin/afl*
/usr/local/bin/afl-analyze  /usr/local/bin/afl-cmin  /usr/local/bin/afl-gcc  /usr/local/bin/afl-showmap
/usr/local/bin/afl-clang    /usr/local/bin/afl-fuzz   /usr/local/bin/afl-gotcpu /usr/local/bin/afl-tmin
/usr/local/bin/afl-clang++  /usr/local/bin/afl-g++    /usr/local/bin/afl-plot  /usr/local/bin/afl-whatsup

```

2. AFL 测试

AFL 可以进行白盒测试，也可以进行黑盒测试。本次复现课本上出现的案例，是一个白盒模糊测试，以理解 AFL 的用法。

2.1>创建测试用程序

新建文件夹 demo，并创建本次实验的程序 Test.c，将源码输入，如下图所示：

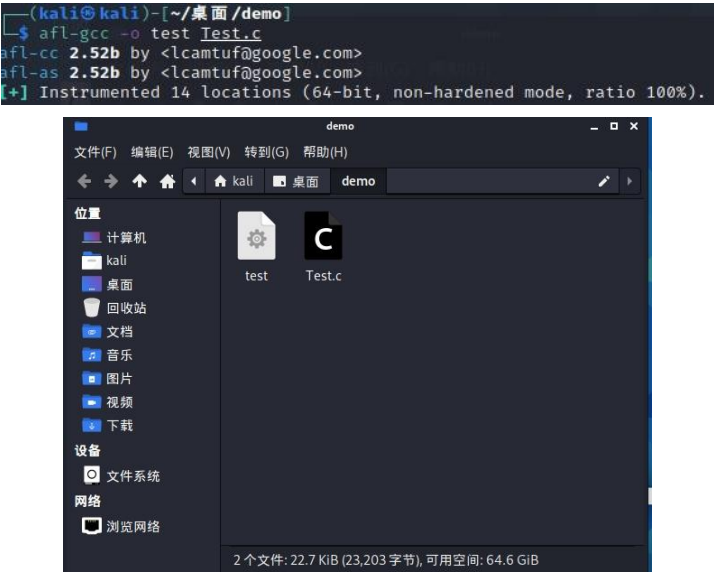
```

#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv) {
    char ptr[20];
    if(argc>1){
        FILE *fp = fopen(argv[1], "r");
        fgets(ptr, sizeof(ptr), fp);
    }
    else{
        fgets(ptr, sizeof(ptr), stdin);
    }
    printf("%s", ptr);
    if(ptr[0] == 'd') {
        if(ptr[1] == 'e') {
            if(ptr[2] == 'a') {
                if(ptr[3] == 'd') {
                    if(ptr[4] == 'b') {
                        if(ptr[5] == 'e') {
                            if(ptr[6] == 'e') {
                                if(ptr[7] == 'f') {
                                    abort();
                                }
                                else printf("%c",ptr[7]);
                            }
                            else printf("%c",ptr[6]);
                        }
                        else printf("%c",ptr[5]);
                    }
                    else printf("%c",ptr[4]);
                }
                else printf("%c",ptr[3]);
            }
            else printf("%c",ptr[2]);
        }
        else printf("%c",ptr[1]);
    }
    else printf("%c",ptr[0]);
    return 0;
}

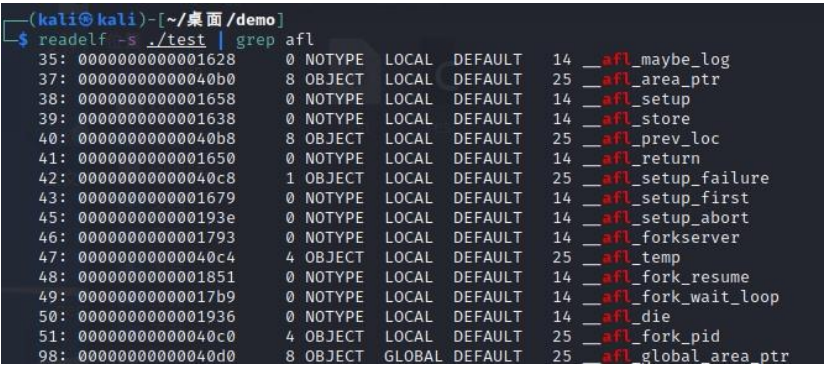
```

可以看出，该代码编译后得到的程序如果被传入“deadbeef”则会终止，如果传入其他字符会原样输出。

使用 afl 的编译器编译，可以使模糊测试过程更加高效。于是输入 `afl-gcc -o test Test.c` 命令行对源代码进行编译，得到一个编译后的文件。如下图所示。



编译后会有插桩符号，用命令 `readelf -s ./test | grep afl` 可以验证，如下图所示：



2.2>创建测试用例

首先，创建两个文件夹 `in` 和 `out`，分别存储模糊测试所需的输入和输出相关的内容。（可以用 `mkdir in out` 命令）



然后，通过 `echo hello> in/foo` 命令行，在输入文件夹中创建一个包含字符串“hello”的文件。如下图所示：



foo 就是我们的测试用例，里面包含初步字符串 hello。AFL 会通过这个语料进行变异，构造更多的测试用例。接下来启动模糊测试即可。

2.3>启动模糊测试

运行 `afl-fuzz -i in -o out -- ./test @@` 命令，启动模糊测试，如下图所示：

```
american fuzzy lop 2.52b (test)

process timing
  run time : 0 days, 0 hrs, 2 min, 25 sec
  last new path : 0 days, 0 hrs, 2 min, 21 sec
  last uniq crash : none seen yet
  last uniq hang : none seen yet
cycle progress
  now processing : 1 (20.00%)
  paths timed out : 0 (0.00%)
stage progress
  now trying : splice 9
  stage execs : 24/32 (75.00%)
  total execs : 183k
  exec speed : 1203/sec
fuzzing strategy yields
  bit flips : 1/160, 1/155, 0/145
  byte flips : 0/20, 0/15, 0/5
  arithmetics : 1/1120, 0/70, 0/0
  known ints : 0/111, 1/397, 0/219
  dictionary : 0/0, 0/0, 0/0
  havoc : 0/92.8k, 0/88.3k
  trim : 33.33%/1, 0.00%

overall results
  cycles done : 50
  total paths : 5
  uniq crashes : 0
  uniq hangs : 0
map coverage
  map density : 0.01% / 0.02%
  count coverage : 1.00 bits/tuple
findings in depth
  favored paths : 5 (100.00%)
  new edges on : 5 (100.00%)
  total crashes : 0 (0 unique)
  total tmouts : 0 (0 unique)
path geometry
  levels : 4
  pending : 0
  pend fav : 0
  own finds : 4
  imported : n/a
  stability : 100.00%

[cpu000: 48%]
```

其中，overall results 一栏可以看到运行的总周期数、总路径数、崩溃次数、超时次数，由上图可见此时还没有检测出崩溃。一般来说，当总期数颜色由洋红色变为黄色、蓝色，最后变成绿色时，可执行的内容已经很少了可以停止测试。这里我们已知有一个漏洞，所以出现一次 crash 就可以 Ctrl-C 停止，如下图所示：

```
american fuzzy lop 2.52b (test)

process timing
  run time : 0 days, 0 hrs, 8 min, 11 sec
  last new path : 0 days, 0 hrs, 0 min, 18 sec
  last uniq crash : 0 days, 0 hrs, 0 min, 6 sec
  last uniq hang : none seen yet
cycle progress
  now processing : 7 (87.50%)
  paths timed out : 0 (0.00%)
stage progress
  now trying : havoc
  stage execs : 7630/12.3k (62.09%)
  total execs : 633k
  exec speed : 1330/sec
fuzzing strategy yields
  bit flips : 4/352, 1/344, 0/328
  byte flips : 0/44, 0/36, 0/20
  arithmetics : 1/2464, 0/70, 0/0
  known ints : 0/244, 1/969, 0/879
  dictionary : 0/0, 0/0, 0/0
  havoc : 1/312k, 0/307k
  trim : 6.67%/4, 0.00%

overall results
  cycles done : 162
  total paths : 8
  uniq crashes : 1
  uniq hangs : 0
map coverage
  map density : 0.02% / 0.03%
  count coverage : 1.00 bits/tuple
findings in depth
  favored paths : 8 (100.00%)
  new edges on : 8 (100.00%)
  total crashes : 1 (1 unique)
  total tmouts : 0 (0 unique)
path geometry
  levels : 7
  pending : 1
  pend fav : 1
  own finds : 7
  imported : n/a
  stability : 100.00%

[cpu000: 47%]

*** Testing aborted by user ***
[+] We're done here. Have a nice day!
```

此时我们打开 out 文件夹里的 crashes 文件夹，可以看到测试保存了产生 crash 的输入，打开发现确实为我们设计的“deadbeef”，验证正确。如下图所示：

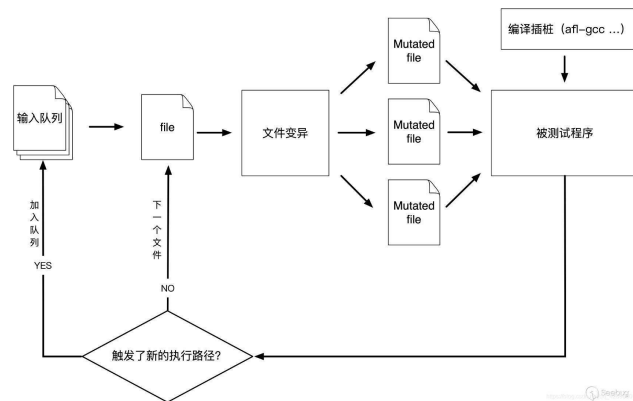


心得体会：

通过实验，深入具体地理解了覆盖引导、文件变异以及 AFL 的用法和工作流程。

AFL 是一款基于覆盖引导的模糊测试工具，它通过记录输入样本的代码覆盖率，从而调整输入样本以提高覆盖率，增加发现漏洞的概率。AFL 主要用于 C/C++ 程序的测试，被测程序有无程序源码均可，有源码时可以对源码进行编译时插桩，无源码可以借助 QEMU 的 User-Mode 模式进行二进制插桩。

其工作流程图如下：



可以总结出工作流程：

- 1 从源码编译程序时进行插桩, 以记录代码覆盖率;
2. 选择一些输入文件, 作为初始测试集加入输入队列;
3. 将队列中的文件按策略进行“突变”;
4. 如果经过变异文件更新了覆盖范围, 则保留在队列中;
5. 循环进行, 期间触发了 crash 的文件会被记录下来。